



UNIVERZITET U NOVOM SADU FAKULTET TEHNIČKIH NAUKA



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
NOVI SAD

PROJEKTNI ZADATAK

Kandidat: Milan Kačarević
Broj indeksa: SV73/2023

Predmet: Objektno orijentisano programiranje 2
Tema rada: Robot u lavirintu u Knososu

Mentor rada: Dušan Stojković

Novi Sad, januar, 2025.

SADRŽAJ

1. Uvod.....	1
1.1 Cilj dokumentacije.....	1
1.2 Pregled problema.....	1
2. Analiza problema.....	3
3. Koncept rješenja.....	4
4. Opis rješenja.....	5
4.1 Rad U/I podсистema (učitavanje i upis/ispis).....	5
4.2 Klase, atributi i funkcije članice.....	5
4.2.1 Ulazna tačka programa (main).....	5
4.2.2 Klasa za postavljanje ćelije u matrici (Cell).....	5
4.2.2.1 Konstruktor za postavljanje tipa ćelije (Cell).....	6
4.2.2.2 Funkcije get i set.....	6
4.2.3 Klasa za osnovne operacije sa matricom (Matrix).....	6
4.2.3.1 Konstruktor za kreiranje matrice i inicijalizaciju parametara (Matrix).....	6
4.2.3.2 Destruktor za oslobađanje memorije zauzete kreiranjem matrice (~Matrix).....	6
4.2.3.3 Funkcija za kreiranje lavirinta i postavljanje elemenata (generateMaze).....	7
4.2.3.4 Funkcija za inicijalizaciju matrice (initializeMatrix).....	7
4.2.3.5 Funkcija za postavljanje spoljašnjih zidova matrice (setExteriorWalls).....	7
4.2.3.6 Funkcija za postavljanje ulaza i izlaza (setEntranceAndExit).....	7
4.2.3.7 Funkcija za računanje optimalnog broja zidova (calculateOptimalWalls).....	7
4.2.3.8 Funkcija za računanje broja predmeta (calculateOptimalNumOfItems).....	7
4.2.3.9 Funkcija za računanje maksimalnog broja zidova (calculateMaxWalls).....	8

4.2.3.10	Funkcija za generisanje unutrašnjih zidova (generateInnerWalls).....	8
4.2.3.11	Funkcija za provjeru povezanosti ulaza i izlaza (hasPath).....	8
4.2.3.12	Funkcija za traženje kolone izlaza (findExitColumn).....	9
4.2.3.13	Funkcija za kreiranje matrice praćenja posjeta (allocateVisitedMatrix).....	9
4.2.3.14	Funkcija za oslobađanje memorije matrice posjeta (freeVisitedMatrix).....	9
4.2.3.15	Funkcija za provjeru puta koristeću BFS algoritam (bfsPathExists).....	9
4.2.3.16	Funkcija za provjeru validnosti ćelije (isValidCell).....	9
4.2.3.17	Funkcija za postavljanje robota, minotaura i predmeta (setItems).....	10
4.2.3.18	Funkcija za postavljanje minotaura (setMinotaurPosition).....	10
4.2.3.19	Funkcija za postavljanje predmeta (placeSpecialItems).....	10
4.2.3.20	Funkcije get i set.....	10
4.2.4	Klasa za prikaz lavirinta (MatrixRenderer).....	11
4.2.4.1	Funkcija za prikazivanje matrice (displayMatrix).....	11
4.2.4.2	Funkcija za prikazivanje magle rata (display3x3Matrix).....	11
4.2.5	Klasa za upravljanje igrom u matrici (Game).....	11
4.2.5.1	Konstruktor za inicijalizaciju parametara (Game).....	12
4.2.5.2	Funkcija za upravljanje kretanjem robota (moveRobot).....	12
4.2.5.3	Funkcija za upravljanje kretanjem minotaura (moveMinotaur).....	12
4.2.5.4	Funkcija za provjeru validnosti poteza (isValidMove).....	13
4.2.5.5	Funkcija za obradu specifičnih radnji (handleParticularCells).....	13
4.2.5.6	Funkcija za ažuriranje pozicija robota ili minotaura (updatePosition).....	13
4.2.5.7	Funkcija za upravljanje efekata predmeta (handleItemEffects).....	14
4.2.5.8	Funkcija za provjeru jedenja robota (checkMinotaurEatsRobot).....	14
4.2.5.9	Funkcija za određivanje pokreta minotaura (randomMinotaurMove).....	14
4.2.5.10	Funkcija za traženje indeksa predmeta u kolekciji (findItemIndex).....	14
4.2.5.11	Funkcija za upravljanje igrom (playTheGame).....	15
4.2.5.12	Funkcija za ispisivanje kontrola (displayControls).....	15
4.2.5.13	Funkcija za ispis matrice (displayMatrix).....	15
4.2.5.14	Funkcija za ispis matrice (handleInvalidMove).....	15
4.2.5.15	Funkcije get i set.....	15
4.2.6	Klasa za upravljanje predmetima (Item).....	16
4.2.6.1	Konstruktor za inicijalizaciju parametara (Item).....	16
4.2.6.2	Apstraktna funkcija za aktivaciju predmeta (activate).....	16
4.2.6.3	Get funkcije.....	16
4.2.7	Klase za aktiviranje predmeta(Fog, Sword, Shield, Hammer).....	16

4.2.7.1	Konstruktor za inicijalizaciju parametara (Fog, Sword, Shield, Hammer)	17
4.2.7.2	Funkcija za aktivaciju predmeta (activate)	17
4.2.8	Klasa za pisanje u fajlove (FileManager)	17
4.2.8.1	Funkcija za zapisivanje sadržaja u datoteku (writeToFile)	17
4.3	Strukture argumenata komandne linije i primjer korišćenja	17
4.4	Struktura izlazne datoteke	18
4.5	Napredniji OO koncepti	18
4.6	Analiza vremena izvršenja	18
4.7	Testiranje u slobodnoj formi – zaključci	19
4.8	Uočeni problemi i ograničenja	25
5.	Zaključak	26

SPISAK SLIKA

Slika 1: Struktura izlazne datoteke.....	18
Slika 2: Susret minotaura s robotom – susjedna pozicija, bez predmeta	19
Slika 3: Susret minotaura s robotom – ista pozicija, bez predmeta	20
Slika 4: Susret minotaura s robotom – ista pozicija, predmet - mač.....	20
Slika 5: Susret minotaura s robotom – susjedna pozicija, predmet – mač.....	21
Slika 6: Susret minotaura s robotom – susjedna pozicija, bez mača.....	21
Slika 7: Susret minotaura s robotom – ista pozicija, bez mača.....	22
Slika 8: Susret minotaura s robotom – ista pozicija, predmet – štit.....	22
Slika 9: Susret minotaura s robotom – susjedna pozicija, predmet - štit	23
Slika 10: Susret minotaura s robotom – susjedna pozicija, bez štita	23
Slika 11: Susret minotaura s robotom – iste pozicije, bez štita.....	24
Slika 12: Susret minotaura s robotom, efekat - magla rata	24
Slika 13: Susret minotaura s robotom, efekat - čekić.....	25

SPISAK TABELA

Table 1: Analiza vremena izvršenja.....	18
---	----

1. Uvod

1.1 Cilj dokumentacije

Cilj ove dokumentacije je da pruži detaljan pregled razvoja i implementacije igre "Robot u lavirintu u Knososu". Dokumentacija je namijenjena korisnicima i programerima koji žele da razumiju arhitekturu i funkcionalnosti sistema, kao i da se upoznaju sa ključnim elementima implementacije. Dokumentacija obuhvata:

- Pregled problema koji je rješavan kroz razvoj sistema.
- Detaljnu analizu svih komponenti programa, uključujući klase i strukture, kao i objašnjenje ključnih funkcionalnosti.
- Opis dizajna sistema sa naglaskom na upotrebu komandne linije, struktura podataka i naprednih objektno-orijentisanih koncepata.
- Testiranje sistema u slobodnoj formi, analizu vremena izvršenja i identifikovane probleme i ograničenja.

Dokumentacija je dizajnirana da omogući lako razumijevanje i korišćenje sistema, kao i da služi kao temelj za buduće nadogradnje i održavanje projekta.

1.2 Pregled problema

U okviru ovog projekta, cilj je implementirati lavirint u kome robot treba da pronađe izlaz, uz to izbjegavajući prepreke i opasnosti, kao što je minotaur. Lavirint je dinamički generisan na osnovu zadatih dimenzija, dok robot mora da reaguje na specifične izazove, kao što su specijalni predmeti sa određenim efektima na osnovu kojih je takođe kreiran lavirint. Projektni zadatak podrazumijeva kreiranje sistema za interakciju korisnika sa lavirintom, omogućujući mu da pomijera robota koji reaguje na efekte predmeta u lavirintu, dok se istovremeno odvija nasumično kretanje minotaura. Glavni izazovi u projektu uključuju:

- Generisanje lavirinta sa zidovima, predmetima, robotom, minotaurom i ulazom i izlazom, prema specifičnim pravilima u objektno-orijentisanom dizajnu.

- Implementacija dinamike igre sa nasumičnim pomijeranjem minotaura i kontrolisanim pomijeranjem robota (kreiranje mehanizma za korisnički unos za pomijeranje robota i interakciju sa lavirintom).
- Definisanje i korišćenje specijalnih predmeta sa različitim efektima (npr. magla rata, štit, mač, čekić).
- Implementacija nasljeđivanja za predmete i njihove efekte.

Pored toga, važno je da program bude u potpunosti funkcionalan u svim scenarijima, kao i da obezbijedi tačne rezultate u različitim testnim slučajevima (u slobodnoj formi), uz mjerenje vremena potrebnog za generisanje lavirinta i sačuvavanje prikaza lavirinta u slučajevima kraja igre.

2. Analiza problema

Problem se može podijeliti u nekoliko povezanih oblasti koje je potrebno analizirati i riješiti na način koji omogućava efikasnu implementaciju i funkcionalnost igre.

Prvi izazov je generisanje lavirinta koji mora biti dinamički kreiran prema zadatim dimenzijama. Lavirint mora sadržavati ulaz i izlaz, zidove, specijalne predmete, robota i minotaura. Potrebno je obezbijediti validnu strukturu lavirinta, gdje su ulaz i izlaz povezani prolazima, dok su zidovi i predmeti postavljeni na način koji poštuje pravila igre. Ključno je pronaći algoritam za generisanje lavirinta koji obezbjeđuje povezanost i optimalnu raspodjelu elemenata.

Drugi aspekt analize je kontrola kretanja unutar lavirinta. Robot se pomjera prema korisničkom unosu, dok minotaur nasumično mijenja svoj položaj. Neophodno je odrediti pravila validacije poteza kako bi se spriječilo kretanje kroz zidove ili izlazak van granica lavirinta. Pored toga, potrebno je implementirati mehanizam za nasumično kretanje minotaura, uz obezbjeđenje da se njegov položaj pravilno ažurira i da se detektuje eventualni susret s robotom.

Treći problem je definisanje specijalnih predmeta i njihovih efekata. Predmeti, poput magle rata, mača, štita ili čekića, imaju specifične uloge u igri i utiču na strategiju igrača. Potrebno je analizirati efekte svakog predmeta, njihov uticaj na dinamiku igre i način njihove interakcije s robotom. Ova analiza vodi ka implementaciji u kojoj će nasljeđivanje najviše odgovarati.

Četvrti aspekt je sinhronizacija kretanja i interakcije svih elemenata igre. Potezi robota i minotaura moraju biti sinhronizovani u svakom koraku, dok interakcija robota s lavirintom i predmetima mora biti pravilno obrađena. Ovo zahtijeva implementaciju mehanizama za upravljanje stanjima igre, uključujući provjeru validnosti poteza, ažuriranje pozicija i primjenu efekata predmeta.

Pored funkcionalnih zahtjeva, potrebno je analizirati i efikasnost implementacije. Vrijeme potrebno za generisanje lavirinta treba biti minimizovano, a prikaz igre mora biti jasno organizovan kako bi igrač mogao lako da upravlja robotom i prati stanje igre.

3. Koncept rješenja

Koncept rješenja za implementaciju igre zasnovan je na fleksibilnom pristupu koji omogućava dinamičko kreiranje lavirinta, kontrolu kretanja elemenata unutar njega i interakciju s posebnim predmetima. Prvo, lavirint će se generisati algoritmom koji koristi podatke o dimenzijama lavirinta i parametrima igre, pri čemu se vodi računa da ulaz i izlaz budu povezani, a da posebni predmeti budu raspoređeni kako bi igra bila dinamična. Postojanje puta vrši se BFS algoritmom.

Kretanje elemenata, uključujući robota (igrača) i minotaura (protivnika), realizovano je kroz posebne logičke komponente. Robot će primati korisničke naredbe za kretanje (npr. gore, dole, lijevo, desno) i provjeravati validnost poteza na osnovu trenutnog stanja lavirinta. Minotaur će se kretati nasumično, pri čemu će birati alternativni potez ako prvobitni nije moguć. Susreti između robota i minotaura detektuju se nakon svakog poteza, a igra završava ako robot izgubi ili pobijedi.

Posebni predmeti unutar lavirinta imaju različite efekte koji se aktiviraju pri susretu s robotom. Predmeti se implementiraju kroz hijerarhiju klasa, pri čemu osnovna klasa Item predstavlja predmet, dok izvedene klase definišu specifične efekte. Na primjer, mač omogućava igraču da eliminiše minotaura, štit štiti robota od susreta s minotaurom, čekić omogućava rušenje zida na susjednom polju, a magla rata ograničava vidljivost robota na okolna polja. Ovi efekti su primjenljivi u samo tri poteza.

Prikaz igre biće realizovan tekstualno, omogućavajući igraču da prati stanje lavirinta, položaj robota, minotaura i predmeta. Prikaz će se ažurirati nakon svakog poteza, pružajući jasnu vizualizaciju trenutnog stanja igre. Kod će biti organizovan u klase koji odgovaraju ključnim funkcionalnostima: generisanje lavirinta, prikaz lavirinta, upravljanje kretanjem, upravljanje specijalnim predmetima i sačuvavanje u fajl. Ova arhitektura olakšava održavanje i proširenje funkcionalnosti.

Igra završava kada robot stigne na izlaz, izgubi susret s minotaurom ili po želji korisnika u bilo koje vrijeme trajanja igre. Ovaj koncept rješenja obuhvata sve ključne elemente igre, pružajući stabilnu osnovu za implementaciju i omogućavajući dalja unapređenja i proširenja.

4. Opis rješenja

4.1 Rad U/I podsistema (učitavanje i upis/ispis)

Aplikacija koristi ulazno-izlazne operacije za rad sa podacima, uključujući:

- Unos preko komandne linije: Korisnik unosi tri argumenta:
 - Broj redova matrice,
 - Broj kolona matrice,
 - Broj predmeta.
- Validacija unosa:
 - Svi argumenti moraju biti cijeli brojevi. Dozvoljeni unos je opcionalan i može se mijenjati, ali optimalan unos je za:
 - redove – od 15 pa nadalje,
 - kolone – od 15 pa nadalje,
 - predmete – od 3 do optimalnog broja(4.2.3.8).
 - Program ispisuje poruku greške ako argumenti nisu ispravni ili su van dozvoljenih granica.
- Ispis rezultata: Matrica i dodatne informacije o igri se ispisuju u konzolu.
- Čuvanje rezultata: Prilikom završetka igre, trenutna matrica čuva se u fajl result.txt uz pomoć metode `FileManager::writeToFile(4.2.8)`.

4.2 Klase, atributi i funkcije članice

U ovom projektu su implementirani izuzeci putem try-catch mehanizma radi sigurnog izvršavanja glavne funkcije, a sve funkcije članice su javne. Takođe su implementirane slobodne funkcije za validaciju ulaza.

4.2.1 Ulazna tačka programa (main)

```
int main(int argc, char* argv[]);
```

Glavna funkcija programa. Prikazuje osnovne informacije o programu, pokreće i uvezuje sve blokove i mjeri i prikazuje vrijeme generisanja lavirinta. Provjerava da li su ulazni parametri validni i računa kolonu u koju će se postaviti robot, a da ona nije rubna.

4.2.2 Klasa za postavljanje ćelije u matrici (Cell)

Klasa za generisanje ćelije, kao osnovnog činioca matrice(lavirinta). Atributi su privatni.

Atributi:

`char` type; - vrijednost koja će se prikazati u toj ćeliji

4.2.2.1 Konstruktor za postavljanje tipa ćelije (Cell)

`Cell();`

Konstruktor koji postavlja početni tip ćelije na '.'.

4.2.2.2 Funkcije get i set

`void setType(char type);` - postavlja tip ćelije na prosljeđeni karakter

`char getType();` - vraća trenutni tip ćelije

4.2.3 Klasa za osnovne operacije sa matricom (Matrix)

Klasa za generisanje lavirinta. Svi atributi su privatni.

Atributi:

`int` rows; - broj redova matrice

`int` cols; - broj kolona matrice

`int` noOfItems; - broj predmeta u matrici

`Cell**` matrix; - dvodimenzioni niz pokazivača na objekte klase Cell

`std::vector<Item*>` items; - dinamička lista pokazivača na objekte klase Item

`Item*` currentItem; - pokazivač na trenutni predmet koji je aktivan

Napomena: Klasa 'Matrix' je dio statičke biblioteke ('.lib') naziva 'MatrixLib', koja je integrisana u glavni projekat.

4.2.3.1 Konstruktor za kreiranje matrice i inicijalizaciju parametara (Matrix)

`Matrix(int rows, int cols, int noOfItems);`

Ovaj konstruktor klase Matrix inicijalizuje instancu matrice sa zadatim brojem redova, kolona i brojem elemenata. Kroz inicijalizacionu listu postavlja početne vrijednosti atributa i alocira dinamičku memoriju za dvodimenzionalni niz objekata tipa Cell. Takođe, inicijalizuje svaki red matrice kao novi niz objekata Cell, omogućavajući rukovanje matricom u memoriji.

Parametri:

- `int` rows – broj redova matrice,
- `int` cols – broj kolona matrice,
- `int` noOfItems – broj predmeta matrice.

4.2.3.2 Destruktor za oslobađanje memorije zauzete kreiranjem matrice (~Matrix)

`~Matrix();`

Ovaj destruktor klase Matrix oslobađa dinamički alociranu memoriju koja je prethodno zauzeta za dvodimenzionalni niz matrice.

4.2.3.3 Funkcija za kreiranje lavirinta i postavljanje elemenata (generateMaze)

`void generateMaze(Game& game);`

Funkcija generateMaze klase Matrix kreira lavirint unutar matrice. generateMaze sadrži u sebi pozive funkcija initializeMatrix, setExteriorWalls, setEntranceAndExit, calculateOptimalWalls, generateInnerWalls, setItems.

Parametri:

- `Game& game` - koristi instancu klase `Game` zbog pristupa atributima ili funkcijama klase `Game`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Game` bez pravljenja kopije, čime se štedi memorija.

4.2.3.4 Funkcija za inicijalizaciju matrice (`initializeMatrix`)

`void initializeMatrix();`

Prolazi kroz svaki element dinamički alociranog dvodimenzionalnog niza objekata tipa `Cell` i postavlja njegov tip na karakter `'.'`.

4.2.3.5 Funkcija za postavljanje spoljašnjih zidova matrice (`setExteriorWalls`)

`void setExteriorWalls();`

Prolazi kroz svaki element dinamički alociranog dvodimenzionalnog niza objekata tipa `Cell` i postavlja njegov tip na karakter `'#'`, pod uslovom da je ivica lavirinta(matrice) u pitanju.

4.2.3.6 Funkcija za postavljanje ulaza i izlaza (`setEntranceAndExit`)

`void setEntranceAndExit(Game& game);`

Funkcija `setEntranceAndExit` postavlja ulaz i izlaz u lavirint. Ulaz se postavlja u prvu vrstu, u kolonu određenu metodom `game.getRobotCol()`(računa se u ulaznoj tački programa-main), dok se izlaz postavlja u posljednju vrstu na nasumično odabranu kolonu koja nije rubna. Ovim se definišu ključne tačke lavirinta potrebne za kretanje kroz njega.

Parametri:

- `Game& game` - koristi instancu klase `Game` zbog pristupa atributima ili funkcijama klase `Game`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Game` bez pravljenja kopije, čime se štedi memorija.

4.2.3.7 Funkcija za računanje optimalnog broja zidova (`calculateOptimalWalls`)

`int calculateOptimalWalls();`

Izračunava optimalan broj unutrašnjih zidova u lavirintu kako bi osigurao balans između kompleksnosti i prohodnosti.

Povratne vrijednosti:

- vraća aritmetičku sredinu minimalnog(zadato u specifikaciji projekta) i maksimalnog(4.2.3.9) broja zidova

4.2.3.8 Funkcija za računanje broja predmeta (`calculateOptimalNumOfItems`)

`int calculateOptimalNumOfItems();`

Funkcija `calculateOptimalNumOfItems` izračunava optimalan broj predmeta koji se mogu smjestiti u matricu na osnovu njene veličine i raspoloživog prostora.

Povratne vrijednosti:

- vraća optimalan broj predmeta koji bi mogao da se smjesti u matricu – to je maksimalan broj predmeta koji korisnik može da unese

4.2.3.9 Funkcija za računanje maksimalnog broja zidova (`calculateMaxWalls`)

`int calculateMaxWalls(int rows, int cols);`

Izračunava maksimalan broj zidova koji se mogu postaviti u matricu na osnovu njene veličine i minimalne dužine puta.

Parametri:

- `int rows` – broj redova matrice,
- `int cols` – broj kolona matrice.

Povratne vrijednosti:

- vraća maksimalan broj zidova koji bi mogao da se smjesti u unutrašnjost lavirinta.

4.2.3.10 Funkcija za generisanje unutrašnjih zidova (`generateInnerWalls`)

```
bool generateInnerWalls(Game& game, int remainingWalls);
```

Funkcija `generateInnerWalls` generiše unutrašnje zidove u lavirintu pokušavajući da postavi zidove na nasumične pozicije dok ne postigne željeni broj preostalih zidova (`remainingWalls`). Svaki put kad se postave svi zidovi, provjerava se da li je lavirint i dalje prohodan korišćenjem metoda `hasPath` (4.2.3.11 - postoji put od ulaza do izlaza).

Parametri:

- `Game& game` - koristi instancu klase `Game` zbog pristupa atributima ili funkcijama klase `Game`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Game` bez pravljenja kopije, čime se štedi memorija,
- `int remainingWalls` – željeni broj unutrašnjih zidova. Jedan od uslova je da on bude 0, jer bi to značilo da su svi zidovi postavljeni (4.2.3.7).

Povratne vrijednosti:

- Metoda je dizajnirana tako da garantuje uspješno generisanje unutrašnjih zidova sa validnim putem kroz matricu. Iako može da ponavlja proces resetovanja i postavljanja zidova više puta (zavisno od slučajnosti), logika osigurava da će prije ili kasnije funkcija `hasPath` vratiti `true`, nakon čega metoda izlazi iz petlje.

4.2.3.11 Funkcija za provjeru povezanosti ulaza i izlaza (`hasPath`)

```
bool hasPath(Game& game);
```

Funkcija `hasPath` provjerava da li postoji put od ulaza do izlaza unutar matrice pomoću BFS algoritma.

Parametri:

- `Game& game` - koristi instancu klase `Game` zbog pristupa atributima ili funkcijama klase `Game`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Game` bez pravljenja kopije, čime se štedi memorija.

Povratne vrijednosti:

- vraća rezultat provjere postojanja puta (`true` ili `false`).

4.2.3.12 Funkcija za traženje kolone izlaza (findExitColumn)

```
int findExitColumn();
```

Ova funkcija traži broj kolone izlaza. Koristi se u hasPath metodi(4.2.3.11)

Povratne vrijednosti:

- vraća broj kolone, u suprotnom -1.

4.2.3.13 Funkcija za kreiranje matrice praćenja posjeta (allocateVisitedMatrix)

```
bool** allocateVisitedMatrix();
```

Dinamički alocira memoriju za dvodimenzionalni niz objekata tipa bool koji se koristi u BFS algoritmu(konkretno u funkcijama hasPath i isFullyConnected)

Povratna vrijednost:

- vraća dinamički alociran dvodimenzionalni niz objekata tipa bool.

4.2.3.14 Funkcija za oslobađanje memorije matrice posjeta (freeVisitedMatrix)

```
void freeVisitedMatrix(bool** visited);
```

Oslobađa dinamički alociranu memoriju koja je prethodno zauzeta za dvodimenzionalni niz(4.2.3.13).

Parametri:

- bool** visited – dinamički alociran dvodimenzionalni niz objekata tipa bool.

4.2.3.15 Funkcija za provjeru puta koristeću BFS algoritam (bfsPathExists)

```
bool bfsPathExists(int entranceRow, int entranceCol, int exitRow, int exitCol, bool** visited);
```

Koristi pretragu u širinu (BFS) kako bi provjerila da li postoji put između ulaza i izlaza u matrici, počinjući od ulaznih koordinata i istražujući susjedne ćelije. Koristi se u funkciji hasPath(4.2.3.11).

Parametri:

- int entranceRow – broj ulaznog reda,
- int entranceCol – broj ulazne kolone,
- int exitRow – broj izlaznog reda,
- int exitCol – broj izlazne kolone,
- bool** visited - dinamički alociran dvodimenzionalni niz objekata tipa bool.

Povratne vrijednosti:

- Ako se pronađe izlaz tokom pretrage, funkcija vraća true, inače vraća false kada pretraga završi bez uspjeha.

4.2.3.16 Funkcija za provjeru validnosti ćelije (isValidCell)

```
bool isValidCell(int newRow, int newCol, bool** visited);
```

Funkcija isValidCell provjerava da li je ćelija unutar granica matrice, nije posjećena, i nije zid. Koristi se u funkciji bfsPathExists(4.2.3.15).

Parametri:

- int newRow – naredni red ćelije,
- int newCol – naredna kolona ćelije,
- bool** visited - dinamički alociran dvodimenzionalni niz objekata tipa bool.

Povratna vrijednost:

- true – ćelija je validna ili false – ćelija nije validna.

4.2.3.17 Funkcija za postavljanje robota, minotaura i predmeta (setlItems)

```
void setlItems(Game& game);
```

Postavlja robota na konkretnu poziciju izračunatu u ulaznoj tački programa(main), a za postavljanje pozicije minotaura i specijalnih efekata(predmeta) koristi funkcije `setMinotaurPosition(4.2.3.18)` i `placeSpecialItems(4.2.3.19)`.

Parametri:

- `Game& game` - koristi instancu klase `Game` zbog pristupa atributima ili funkcijama klase `Game`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Game` bez pravljenja kopije, čime se štedi memorija.

4.2.3.18 Funkcija za postavljanje minotaura (`setMinotaurPosition`)

`void setMinotaurPosition(Game& game);`

Funkcija koja postavlja minotaura na random validnu poziciju u početku igre. Da bi robot mogao da se kreće u početku igre, minotaur će uvijek biti postavljen na bilo koju validnu poziciju u donjoj polovini lavirinta.

Parametri:

- `Game& game` - koristi instancu klase `Game` zbog pristupa atributima ili funkcijama klase `Game`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Game` bez pravljenja kopije, čime se štedi memorija.

4.2.3.19 Funkcija za postavljanje predmeta (`placeSpecialItems`)

`void placeSpecialItems(Game& game);`

Funkcija koja će na random validnim pozicijama u lavirintu postaviti predmete uz random biranje vrste specijalnog efekta za svaki predmet.

Parametri:

- `Game& game` - koristi instancu klase `Game` zbog pristupa atributima ili funkcijama klase `Game`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Game` bez pravljenja kopije, čime se štedi memorija.

4.2.3.20 Funkcije `get` i `set`

`Cell** getMatrix();` - vraća pokazivač na matricu.

`void setMatrix(Cell** matrix);` - postavlja matricu na zadatu vrijednost

`int getMatrixRows();` - vraća broj redova matrice

`int getMatrixCols();` - vraća broj kolona matrice

`int getNoOfItems();` - vraća broj predmeta

`Item* getCurrentItem();` - vraća trenutno aktivan predmet

`void setCurrentItem(Item* item);` - postavlja trenutno aktivan predmet

`std::vector<Item*> &getItems();` - vraća referencu na kolekciju predmeta

`void setItems(Item* item);` - dodaje predmet u kolekciju predmeta

4.2.4 Klasa za prikaz lavirinta (`MatrixRenderer`)

Klasa koja sadrži metode za prikazivanje lavirinta. Nema attribute.

Napomena: Klasa `MatrixRenderer` je dio statičke biblioteke (`.lib`) naziva `MatrixLib`, koja je integrisana u glavni projekat.

4.2.4.1 Funkcija za prikazivanje matrice (displayMatrix)

```
void displayMatrix(Matrix& matrix, bool useColor, std::ostream& os = std::cout);
```

Prikazuje trenutno stanje lavirinta. Statička je funkcija(ne mora se kreirati objekat klase da bi se funkcija pozvala).

Parametri:

- Matrix& matrix – koristi instancu klase Matrix zbog pristupa atributima ili funkcijama klase Matrix. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Matrix bez pravljenja kopije, čime se štedi memorija,
- bool useColor – da li će lavirint biti prikazan u boji ili ne,
- std::ostream& os = std::cout - Referenca na izlazni tok (tipa std::ostream) kroz koji se matrica prikazuje. Ovo omogućava fleksibilnost pri odabiru gdje će se matrica prikazati, na primjer, na standardni izlaz (std::cout) ili u fajl.

4.2.4.2 Funkcija za prikazivanje magle rata (display3x3Matrix)

```
void display3x3Matrix(Matrix& matrix, bool useColor, Game& game, std::ostream& os = std::cout);
```

Prikazuje matricu u slučaju aktiviranog specijalnog efekta – magla rata. Statička je funkcija(ne mora se kreirati objekat klase da bi se funkcija pozvala).

Parametri:

- Matrix& matrix – koristi instancu klase Matrix zbog pristupa atributima ili funkcijama klase Matrix. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Matrix bez pravljenja kopije, čime se štedi memorija,
- bool useColor – da li će lavirint biti prikazan u boji ili ne,
- Game& game - koristi instancu klase Game zbog pristupa atributima ili funkcijama klase Game. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Game bez pravljenja kopije, čime se štedi memorija,
- std::ostream& os = std::cout - Referenca na izlazni tok (tipa std::ostream) kroz koji se matrica prikazuje. Ovo omogućava fleksibilnost pri odabiru gdje će se matrica prikazati, na primjer, na standardni izlaz (std::cout) ili u fajl.

4.2.5 Klasa za upravljanje igrom u matrici (Game)

Klasa koji sadrži funkcije upravljanja igrom, kretanja robota i minotaura.

Atributi(privatni):

```
int robotRow, robotCol; - trenutna pozicija robota
int robotNewRow, robotNewCol; - naredna pozicija robota
int minotaurRow, minotaurCol; - trenutna pozicija minotaura
int itemDuration; - trajanje predmeta
```

Atributi(javni):

```
bool isFog; - da li je efekat magle rata aktivna
bool hasSword; - da li je efekat mača aktivan
bool minotaurKilled; - da li je minotaur ubijen
bool hasShield; - da li je efekat štita aktivan
bool hasHammer; - da li je efekat čekića aktivan
```

4.2.5.1 Konstruktor za inicijalizaciju parametara (Game)

```
Game(int robotRow, int robotCol, int minotaurRow, int minotaurCol);
```

Konstruktor klase Game inicijalizuje pozicije robota i minotaura, zajedno sa početnim vrijednostima ostalih atributa igre.

Parametri:

- int robotRow – početna koordinata reda Robota
- int robotCol – početna koordinata kolone Robota
- int minotaurRow - početna koordinata reda Minotaura
- int minotaurCol - početna koordinata kolone Minotaura

4.2.5.2 Funkcija za upravljanje kretanjem robota (moveRobot)

```
bool moveRobot(Matrix& matrix, char direction);
```

Ova funkcija omogućava pomjeranje robota unutar igre u zavisnosti od unesenog smjera ('w', 's', 'a', 'd'). Prvo određuje promjene u koordinatama na osnovu smjera, zatim provjerava da li je potez validan – uzimajući u obzir prisustvo čekića i tip prepreke. Ako je potez dozvoljen, ažurira poziciju robota, obrađuje specifične ćelije na koje robot stupa i ažurira matricu igre. Na kraju, vraća true ako je potez uspješno izvršen, ili false ako nije.

Parametri:

- Matrix& matrix – koristi instancu klase Matrix zbog pristupa atributima ili funkcijama klase Matrix. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Matrix bez pravljenja kopije, čime se štedi memorija,
- char direction – karakter koji označava smjer u kojem robot treba da se pomjeri ('w' za gore, 's' za dole, 'a' za lijevo, 'd' za desno).

Povratna vrijednost:

- true – potez uspješan,
- false – potez neuspješan.

4.2.5.3 Funkcija za upravljanje kretanjem minotaura (moveMinotaur)

```
bool moveMinotaur(Matrix& matrix);
```

Ova funkcija upravlja kretanjem minotaura u igri, provjerava da li je minotaur uništio robota i upravlja efektima predmeta ako su aktivni.

Parametri:

- Matrix& matrix – koristi instancu klase Matrix zbog pristupa atributima ili funkcijama klase Matrix. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Matrix bez pravljenja kopije, čime se štedi memorija.

Povratne vrijednosti:

- Ako je minotaur ubijen, funkcija odmah vraća true. Zatim, provjerava da li je minotaur pojeo robota i u tom slučaju vraća false. Ako to nije slučaj, minotaur izvršava slučajni potez u matrici nakon upravljanja efektima i nakon toga funkcija vraća true kako bi označila da je potez uspješno izvršen.

4.2.5.4 Funkcija za provjeru validnosti poteza (isValidMove)

```
bool isValidMove(Matrix& matrix, int newRow, int newCol);
```

Provjerava da li je kretanje na zadanu poziciju u matrici dozvoljeno.

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija,
- `int newRow` – red potencijalne nove pozicije,
- `int newCol` – kolona potencijalne nove pozicije.

Povratna vrijednost:

- `true` – potez je validan,
- `false` – potez nije validan.

4.2.5.5 Funkcija za obradu specifičnih radnji (`handleParticularCells`)

```
void handleParticularCells(Matrix& matrix, int newRow, int newCol);
```

Funkcija `handleParticularCells` obrađuje specifične radnje koje se dešavaju kada robot uđe u ćeliju određene vrste. Ako robot stigne na ćeliju izlaza, igra se završava pobjedom i rezultat se zapisuje u datoteku. Ako robot uđe u ćeliju s predmetom, ažuriraju se odgovarajući efekti predmeta, poput aktiviranja magle, mača, štita ili čekića.

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija,
- `int newRow` – red u matrici na koji robot pokušava da se pomjeri,
- `int newCol` – kolona u matrici na koju robot pokušava da se pomjeri.

4.2.5.6 Funkcija za ažuriranje pozicija robota ili minotaura (`updatePosition`)

```
void updatePosition(Matrix& matrix, int oldRow, int oldCol, int newRow, int newCol, char type);
```

Ažurira položaj robota ili Minotaura u matrici igre.

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija,
- `int oldRow` – redni broj reda u kojem se robot ili minotaur nalazio prije pomjeranja,
- `int oldCol` – redni broj kolone u kojoj se robot ili minotaur nalazio prije pomjeranja,
- `int newRow` – redni broj reda u koji se robot ili minotaur pomjera,
- `int newCol` – redni broj kolone u koji se robot ili minotaur pomjera,
- `char type` – tip ('R' – robot ili 'M' – minotaur) koji se pomjera.

4.2.5.7 Funkcija za upravljanje efektima predmeta (`handleItemEffects`)

```
void handleItemEffects(Matrix& matrix);
```

Upravlja efektima predmeta koje robot posjeduje. Ako je trajanje efekta predmeta isteklo efekti se deaktiviraju postavljanjem atributa (`isFog`, `hasSword`, `hasShield`, `hasHammer`) na `false`. U suprotnom, smanjuje trajanje efekta za jedan korak i aktivira trenutni predmet na osnovu

njegovog specifičnog efekta pozivom funkcije `activate` predmeta. Efekat magle rata i čekića traju tri poteza nakon dobijanja predmeta, a efekti štita i mača traju u trenutku dobijanja predmeta i naredna dva poteza, ukupno tri.

Parametri:

- `Matrix& matrix` - koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija.

4.2.5.8 Funkcija za provjeru jedenja robota (`checkMinotaurEatsRobot`)

`bool checkMinotaurEatsRobot(Matrix& matrix);`

Provjerava da li je minotaur pojeo robota provjerom susjednih polja minotaura.

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija.

Povratne vrijednosti:

- `true` – minotaur je pojeo robota,
- `false` – minotaur nije pojeo robota.

4.2.5.9 Funkcija za određivanje pokreta minotaura (`randomMinotaurMove`)

`void randomMinotaurMove(Matrix& matrix, int exitRow, int exitCol);`

Nasumično pomjera Minotaura unutar matrice igre, vodeći računa o validnosti poteza.

Koristi se u metodi `moveMinotaur`(4.2.5.3).

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija,
- `int exitRow` – red u kojem se nalazi izlaz iz matrice, koji Minotaur ne smije dostići,
- `int exitCol` – kolona u kojoj se nalazi izlaz iz matrice, koju Minotaur ne smije dostići.

4.2.5.10 Funkcija za traženje indeksa predmeta u kolekciji (`findItemIndex`)

`int findItemIndex(Matrix& matrix, int row, int col) const;`

Pronalazi indeks predmeta (`item`) u kolekciji na osnovu prosljeđenih koordinata reda i kolone. Koristi se u metodi `handleParticularCells`(4.2.5.5).

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija,
- `int row` – red matrice na kojem se možda nalazi taj predmet,
- `int col` – kolona matrice na kojoj se možda nalazi taj predmet.

Povratne vrijednosti:

- vraća indeks predmeta u kolekciji predmeta ako postoji, a ako ne postoji vraća -1.

4.2.5.11 Funkcija za upravljanje igrom (playTheGame)

`void playTheGame(Matrix& matrix);`

Upravlja glavnom petljom igre i omogućava korisniku interakciju s igrom.

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija.

4.2.5.12 Funkcija za ispisivanje kontrola (displayControls)

`void displayControls();`

Ispisuje na ekranu moguće komande koje korisnik može da unese da odigrao potez.

4.2.5.13 Funkcija za ispis matrice (displayMatrix)

`void displayMatrix(Matrix& matrix);`

Radi čitkosti koda ova funkcija posebno provjerava da li ne traje magla rata i poziva metodu za ispisivanje matrice iz klase `Maze` u klasi `Game`.

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija.

4.2.5.14 Funkcija za ispis matrice (handleInvalidMove)

`void handleInvalidMove(Matrix& matrix);`

Radi čitkosti koda ova funkcija posebno provjerava da li traje magla rata i posebno ispisuje matricu u slučaju da tokom trajanja ili netrajanja magle rata tokom nevalidnog poteza ispiše poruku. Koristi se u metodi `playTheGame`(4.2.5.11).

Parametri:

- `Matrix& matrix` – koristi instancu klase `Matrix` zbog pristupa atributima ili funkcijama klase `Matrix`. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase `Matrix` bez pravljenja kopije, čime se štedi memorija.

4.2.5.15 Funkcije get i set

Za attribute ove klase implementirane su `get` i `set` funkcije. `Set` funkcije omogućavaju postavljanje atributa na zadatu vrijednost koja se proslijeđuje kao parametar, dok `get` funkcije vraćaju trenutnu vrijednost atributa.

```
int getRobotRow();
void setRobotRow(int row);
int getRobotCol();
void setRobotCol(int col);
int getRobotNewRow();
void setRobotNewRow(int row);
int getRobotNewCol();
void setRobotNewCol(int col);
int getMinotaurRow();
void setMinotaurRow(int row);
int getMinotaurCol();
void setMinotaurCol(int col);
int getItemDuration();
void setItemDuration(int itemDuration);
```

4.2.6 Klasa za upravljanje predmetima (Item)

Item predstavlja apstraktnu baznu klasu za različite vrste predmeta koji se nalaze u matrici igre. Njena glavna svrha je definisanje osnovnih atributa i funkcija koje specifični predmeti moraju implementirati. Ova klasa koristi mehanizam polimorfizma za omogućavanje različitog ponašanja specifičnih predmeta kroz virtuelnu funkciju activate. Svi atributi su zaštićeni.

Atributi:

`int` row, col; - pozicija predmeta
`char` type; - tip predmeta

4.2.6.1 Konstruktor za inicijalizaciju parametara (Item)

```
Item(int r, int c, char t);
```

Konstruktor klase Item inicijalizuje red, kolonu i tip predmeta.

Parametri:

- `int r` – broj reda predmeta,
- `int c` – broj kolone predmeta,
- `int t` – tip predmeta.

4.2.6.2 Apstraktna funkcija za aktivaciju predmeta (activate)

```
virtual void activate(Matrix& matrix, Game& game) = 0;
```

Čista virtuelna funkcija koja mora biti implementirana u izvedenim klasama.

Parametri:

- `Matrix& matrix` – koristi instancu klase Matrix zbog pristupa atributima ili funkcijama klase Matrix. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Matrix bez pravljenja kopije, čime se štedi memorija,
- `Game& game` – koristi instancu klase Game zbog pristupa atributima ili funkcijama klase Game. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Game bez pravljenja kopije, čime se štedi memorija.

4.2.6.3 Get funkcije

Implementirane get funkcije vraćaju trenutne vrijednosti atributa.

```
int getRow();  
int getCol();  
char getType();
```

4.2.7 Klase za aktiviranje predmeta(Fog, Sword, Shield, Hammer)

Klasa Fog(Sword, Shield ili Hammer) je izvedena iz klase Item(4.2.6), a u konstruktoru koristi inicijalizaciju roditeljske klase sa parametrima `r` i `c` (red i kolona) i karakterom 'F', 'S', 'E' ili 'H', što ukazuje da ovaj objekat predstavlja redom, maglu rata, mač, štit ili čekić.

4.2.7.1 Konstruktor za inicijalizaciju parametara (Fog, Sword, Shield, Hammer)

```
Fog(int r, int c);  
Sword(int r, int c);  
Shield(int r, int c);  
Hammer(int r, int c);
```

Konstruktor poziva konstruktor roditeljske klase Item, što znači da Fog nasljeđuje sve osobine klase Item, uključujući njene funkcije i attribute. Karakter tipa predmeta se automatski postavlja(implicitno), što omogućuje da se u konstruktoru ovih klasa nalaze samo dva parametra, Parametri:

- int r – broj reda predmeta,
- int c – broj kolone predmeta.

4.2.7.2 Funkcija za aktivaciju predmeta (activate)

`void activate(Matrix& matrix, Game& game) override;`

Odgovorna je za aktivaciju efekta magle rata, mača, štita ili čekića. Override se koristi kako bi se označilo da funkcija activate nadmašuje odgovarajuću metodu iz roditeljske klase.

Parametri:

- Matrix& matrix – koristi instancu klase Matrix zbog pristupa atributima ili funkcijama klase Matrix. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Matrix bez pravljenja kopije, čime se štedi memorija,
- Game& game – koristi instancu klase Game zbog pristupa atributima ili funkcijama klase Game. Razlog korišćenja reference leži u direktnom pristupu originalnom objektu klase Game bez pravljenja kopije, čime se štedi memorija.

4.2.8 Klasa za pisanje u fajlove (FileManager)

Služi za zapisivanje matrice u fajl u slučaju završetka igre.

4.2.8.1 Funkcija za zapisivanje sadržaja u datoteku (writeToFile)

`static void writeToFile(const std::string& filename, const std::string& content);`

Služi za zapisivanje sadržaja u datoteku. Statička je funkcija(ne mora se kreirati objekat klase da bi se funkcija pozvala).

Parametri:

- const std::string& filename – referenca na konstantni objekat tipa std::string koji predstavlja naziv datoteke u koju će sadržaj biti upisan. Korišćenjem reference izbjegava se nepotrebno kopiranje stringa, a const obezbjeđuje da sadržaj naziva datoteke neće biti promijenjen unutar funkcije,
- const std::string& content – referenca na konstantni objekat tipa std::string koji predstavlja tekstualni sadržaj koji treba da bude upisan u datoteku. Kao i kod prethodnog parametra, referenca sprječava kopiranje stringa, a const garantuje da sadržaj neće biti izmijenjen unutar funkcije.

4.3 Strukture argumenata komandne linije i primjer korišćenja

Aplikacija je testirana na Linux operativnom sistemu. Projekat je razvijen u CLion-u i koristi CMake kao build sistem. Program se najprije build-a (Debug ili Release), nakon čega CLion automatski generiše odgovarajući build direktorijum (cmake-build-debug/ ili cmake-build-release/). Za pokretanje programa potrebno je pozicionirati se u app direktorijum unutar build foldera. Aplikacija koristi sljedeći format za unos: `./MazeGame <broj_redova> <broj_kolona> <broj_predmeta>`. Rezultat(result.txt) će se ispisati u app direktorijumu build direktorijuma.

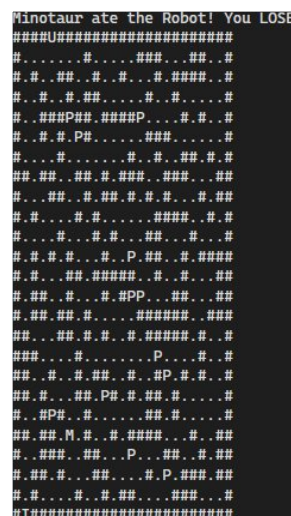
Primjer poziva: `./MazeGame 20 20 30`

Takođe, na samo pokretanje programa u konzoli može automatski da se generiše matrica(Run/Edit Configurations/Program Arguments). Postaviti polje Program Arguments kao “20 20 30”.

4.4 Struktura izlazne datoteke

Rezultati igre se čuvaju u tekstualnu datoteku result.txt. Struktura datoteke je sljedeća:

- Naslov: Kratak opis stanja igre (razlog završetka igre).
- Matrica: Tekstualna reprezentacija matrice generisane tokom igre.



Slika 1: Struktura izlazne datoteke

4.5 Napredniji OO koncepti

U implementaciji klase Item(4.2.6) korišteni su napredni koncepti objektno orijentisanog programiranja, uključujući nasljeđivanje i polimorfizam(ista metoda se ponaša različito u zavisnosti od tipa objekta koji je koristi). Korišćenje pristupa "apstraktne bazne klase" osigurava fleksibilnost i proširivost sistema, jer nove vrste predmeta mogu biti dodane jednostavno nasljeđivanjem Item klase, prosljeđivanjem odgovarajućih argumenata konstruktoru bazne klase i definisanjem vlastite verzije metode activate.

4.6 Analiza vremena izvršenja

Za različite argumente tabelarno je prikazano vrijeme izvršenja (uvijek treba uzeti u obzir da zbog BFS algoritma za potvrdu puta od ulaza do izlaza mogu da budu različita vremena izvršavanja):

Table 1: Analiza vremena izvršenja

Vrijeme	Broj Redova	Broj Kolona	Broj Predmeta
0.0000784s	16	16	21
0.0004051s	25	25	50
0.0005243s	30	20	20
0.0001611s	35	70	20
0.0002714s	37	100	70
0.0002914s	50	70	100
0.0015302s	100	130	1000

0.0047540s	500	150	200
0.0199076s	1500	200	60000

Sva mjerenja su izvršena na osmojezgrenom procesoru AMD Ryzen 7 5700U with Radeon Graphics 1.80 GHz.

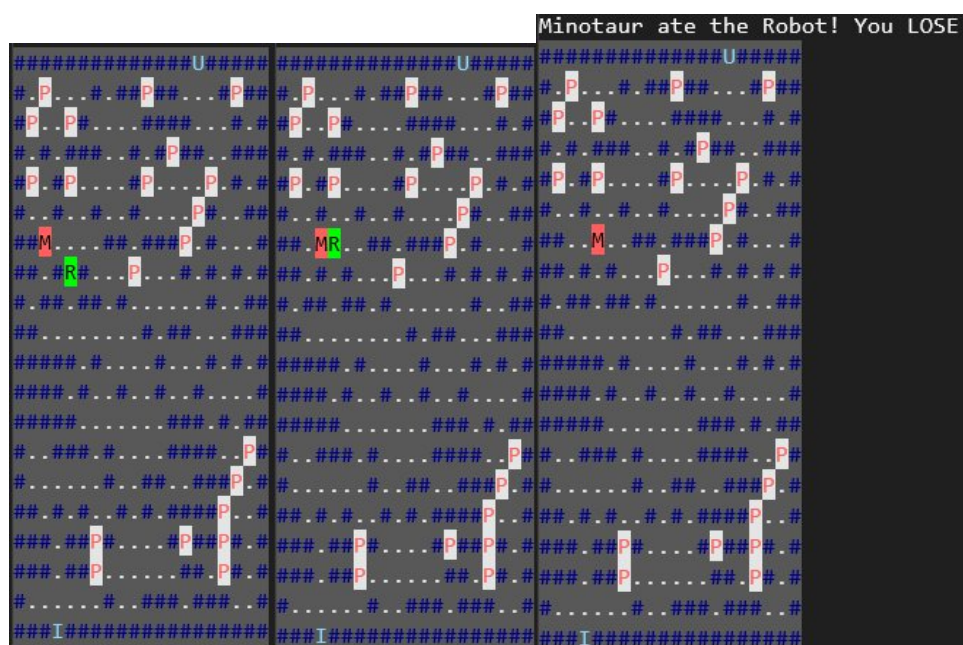
Vremena izvršavanja programa su mjerena u Release modu.

Za mjerenje vremena trajanja generisanja lavirinta u ovoj implementaciji, korišćena je standardna C++11 `std::chrono` biblioteka, koja omogućava precizno mjerenje vremena.

4.7 Testiranje u slobodnoj formi – zaključci

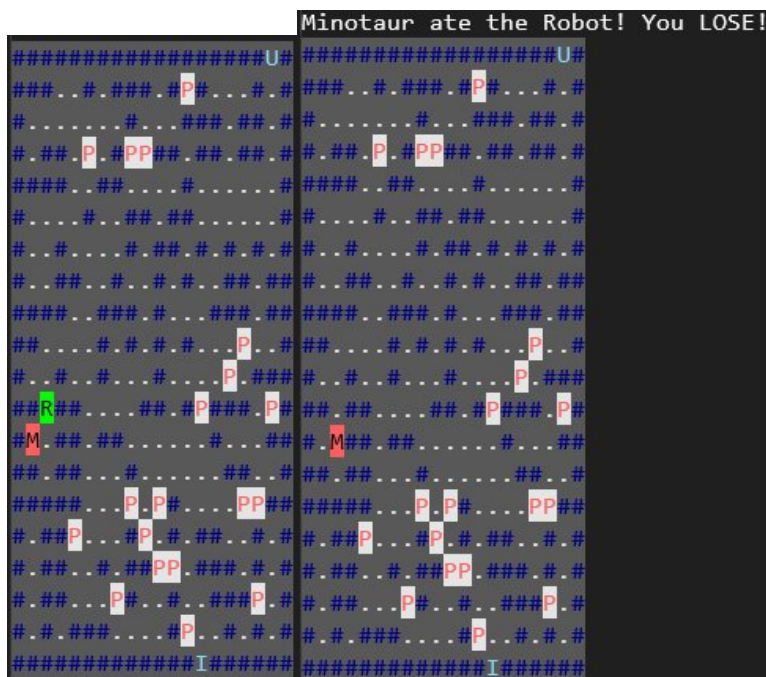
Tokom testiranja kroz igranje ove igrice moglo se zaključiti sljedeće:

- u slučaju bilo kakvog susreta minotaura sa robotom (ili su jedno pored drugog ili odigraju na istu poziciju), a da robot nema štit ili mač, robot je uništen (minotaur će se automatski prebaciti na poziciju robota). Na slici ispod je prikazan slučaj kada minotaur i robot dođu u susjedne pozicije, a da robot nema nikakvu zaštitu ili mač.



Slika 2: Susret minotaura s robotom – susjedna pozicija, bez predmeta

Na slici ispod je prikazan slučaj kada minotaur i robot odaberu istu poziciju, a da robot nema nikakvu zaštitu ili mač.



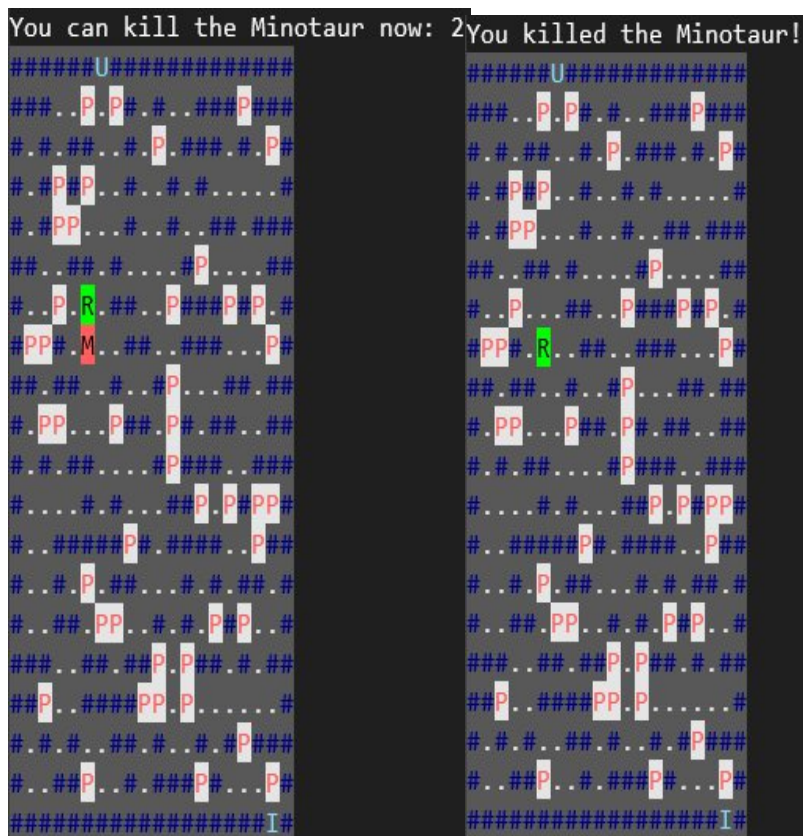
Slika 3: Susret minotaura s robotom – ista pozicija, bez predmeta

- kada korisnik aktivira maglu rata ili čekić prvi put, taj potez i naredna tri poteza će biti u tom modu(mora da prođe tri koraka u modu magle rata, jer potez kojim je dobio maglu rata se ne računa-tada korisnik vidi cijelu matricu i mora da prođe tri koraka u modu magle rata i mora da prođe tri koraka u modu čekića, jer mora da ima mogućnost da uništi tri zida)
- kada korisnik aktivira štit ili mač, taj potez i naredna dva poteza biće u tom modu. Dakle prvi potez štita ili mača aktiviraće te efekte i trajaće naredna dva koraka(ukupno 3).
- postoji slučaj kada robot dobije mač i u trenutku dobijanja tog efekta minotaur se nađe na istoj poziciji.



Slika 4: Susret minotaura s robotom – ista pozicija, predmet - mač

- ako se minotaur nađe na susjednoj poziciji u trenutku kada robot dobije ili ima mač, onda će robot automatski da uništi minotaura.



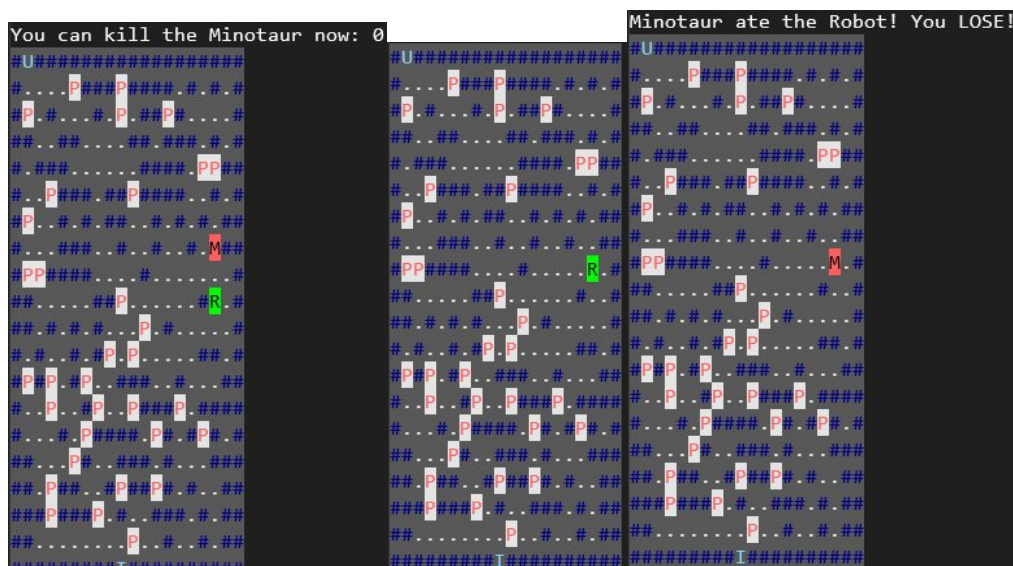
Slika 5: Susret minotaura s robotom – susjedna pozicija, predmet – mač

- u ovom slučaju, robotu je ostalo 0 poteza sa mačem, tako da u susretu sa minotaurom(susjedne pozicije) biva uništen.



Slika 6: Susret minotaura s robotom – susjedna pozicija, bez mača

- u ovom slučaju, robotu je ostalo 0 poteza sa mačem, tako da u susretu sa minotaurom(iste pozicije) biva uništen.



Slika 7: Susret minotaura s robotom – ista pozicija, bez mača

- postoji slučaj kada robot dobije štit i u trenutku dobijanja tog efekta minotaur se nađe na istoj poziciji.



Slika 8: Susret minotaura s robotom – ista pozicija, predmet – štit

- ako se minotaur nađe na susjednoj poziciji u trenutku kada robot dobije ili ima štit, ništa se ne dešava.



Slika 9: Susret minotaura s robotom – susjedna pozicija, predmet - štit

- u ovom slučaju, robotu je ostalo 0 poteza sa štitom, tako da u susretu sa minotaurom(susjedne pozicije) biva uništen.



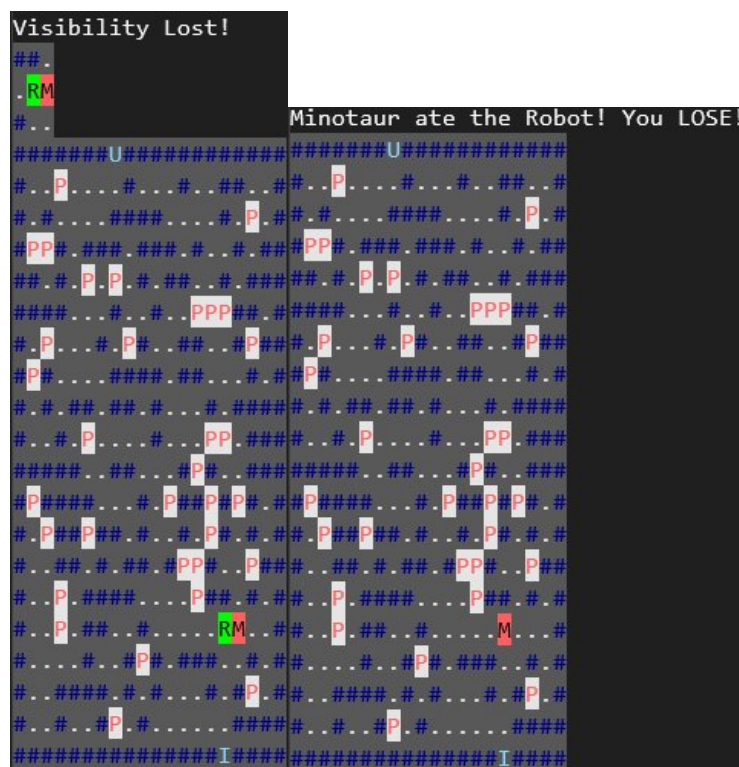
Slika 10: Susret minotaura s robotom – susjedna pozicija, bez štita

- u ovom slučaju, robotu je ostalo 0 poteza sa štitom, tako da u susretu sa minotaurom(iste pozicije) biva uništen.



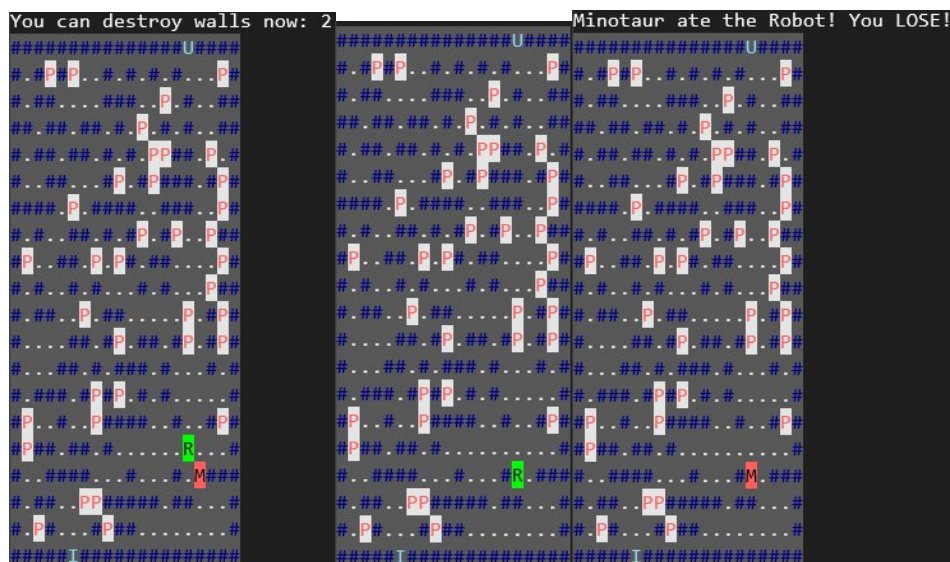
Slika 11: Susret minotaura s robotom – iste pozicije, bez štita

- u slučaju kada je vidljivost izgubljena, moguće je izgubiti igru.



Slika 12: Susret minotaura s robotom, efekat - magla rata

- u slučaju kada robot uzima/ima čekić, minotaur može uništiti robota.



Slika 13: Susret minotaura s robotom, efekat - čekić

4.8 Uočeni problemi i ograničenja

Što je veći lavirint, veća je nepreglednost tokom igranja i manje je korisničko iskustvo. Bilo bi dobro napraviti da algoritam pravi lavirinte koji u sebi ne sadrže konture, kako minotaur ili predmeti ne bi bili u jednoj od njih i kako bi cijeli lavirint bio iskoristiv za igru.

Minotaur može da ima predvidive i nespretne pokrete, što nije cilj same igrice, jer bi trebala biti teža za dolazak do ciljne pozicije. Jedno od poboljšanja bi bilo da se minotaur kreće brže(jedan potez robota je više poteza minotaura) i implementacija pamćenja pozicije robota, ako je on minotauru na vidiku.

5. Zaključak

U ovoj implementaciji igre lavirinta razvijen je funkcionalni model za generisanje lavirinata, kretanje robota i minotaura, kao i interakciju sa specijalnim predmetima. Implementirani su algoritmi za generisanje zidova, postavljanje ulaza i izlaza, kao i logika za postavljanje i aktivaciju predmeta (mačeva, štitova, čekića,...). Sve funkcionalnosti su verifikovane kroz razne pokušaje igranja igre, pri čemu su provjereni različiti scenariji ponašanja, kao što je pravilno postavljanje predmeta, kao i efikasnost generisanja lavirinata.